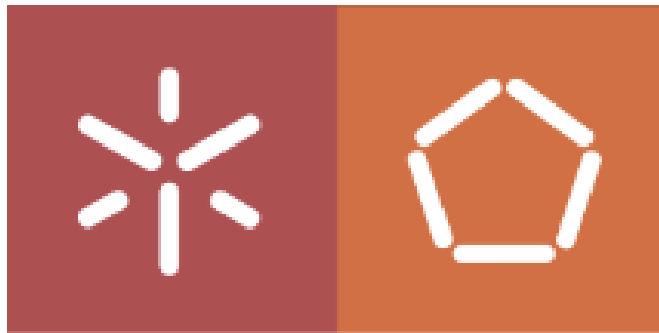


Computação Gráfica

Trabalho Prático: Fase 1

Fevereiro 2026



Universidade do Minho
Escola de Engenharia

Grupo: 8

Realizado por: Juliana Sofia Vaz da Silva, a105572

Sofia Beatriz Miranda Couto, a106925

Soraia Filipa Ribeiro Pereira, a106806

Índice

Introdução	2
1. Generator	3
1.1. Ficheiros de armazenamento	3
1.2. Implementação das Primitivas gráficas	3
1.2.1. Plane	4
1.2.2. Box	4
1.2.3. Sphere	5
1.2.4. Cone	7
1.2.5. Cylinder	9
1.2.6. Torus	10
2. Engine	12
2.1. Parsing e Estrutura de Dados	12
2.1.1. Point	12
2.1.2. Parsing do Ficheiro XML	12
2.1.2.1. Scene	13
2.1.3. Parsing do Ficheiro .3d	13
2.2. Renderização da Cena	13
2.2.1. Configuração de Visualização	13
2.2.2. Desenho Final	13
Conclusão	14
Referências e Bibliografia	15

Introdução

Este relatório descreve o desenvolvimento da primeira fase do projeto da unidade curricular de Computação Gráfica, inserida no 2º semestre do 3º ano da Licenciatura em Engenharia Informática, no ano letivo 2025/2026.

Sendo esta a primeira parte de um projeto dividido em quatro fases, o objetivo principal consiste no desenvolvimento de dois componentes fundamentais: o ***Engine*** e o ***Generator***. Estes componentes serão fundamentais para a implementação das fases seguintes.

1. Generator

O *Generator* é um programa de consola independente, responsável por criar os ficheiros com os vértices necessários ao desenho de uma primitiva gráfica, recebida em argumento.

Inicialmente, são calculados os pontos consoante parâmetros dados à aplicação, sendo guardados num ficheiro, de seguida.

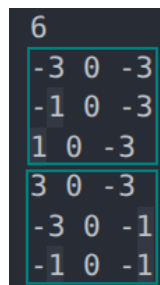
Para gerar o ficheiro de armazenamento utiliza-se um dos seguintes comandos:

```
./generator cone <raio> <altura> <slices> <stacks> cone.3d
./generator sphere <raio> <slices> <stacks> sphere.3d
./generator box <length> <grid> box.3d
./generator plane <length> <divisions> plane.3d
./generator cylinder <raio> <altura> <slices> cylinder.3d
./generator torus <raio_maior> <raio_menor> <stacks> <slices> torus.3d
```

1.1. Ficheiros de armazenamento

O cálculo para todas as primitivas utiliza triângulos para a aproximação das superfícies geométricas. Para assegurar a correta visualização dos modelos, os vértices de cada triângulos são gerados por uma ordem específica, permitindo a correta aplicação de técnicas como o *Backface Culling*.

Uma vez calculados, estes pontos são armazenados num ficheiro com extensão *.3d*, onde a primeira linha indica o número total de vértices necessários, seguidos das coordenadas cartesianas (*x*, *y*, *z*) dos pontos, um por linha. A organização dos pontos foi pensada de forma a que cada grupo de três linhas define um triângulo, como destacado a azul na Figura 1.



```
6
-3 0 -3
-1 0 -3
1 0 -3
3 0 -3
-3 0 -1
-1 0 -1
```

Figura 1 - Exemplo de um ficheiro com extensão *.3d*

1.2. Implementação das Primitivas gráficas

Para compreender a implementação de algumas das primitivas, é necessário, primeiro, perceber o que são *slices* e *stacks* na representação de um sólido.

- **Slices**: divisões verticais no sólido, como destacado a azul na figura 2.
- **Stacks**: divisões horizontais no sólido, como destacado a laranja na figura 2.

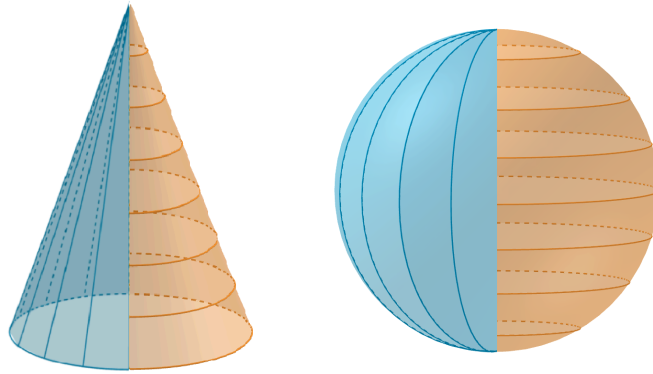


Figura 2 - Representação de slices e stacks em diferentes sólidos

1.2.1. *Plane*

A primitiva *Plane* representa um quadrado desenhado no plano xz , centrado na origem. A sua construção depende de dois parâmetros: o comprimento da aresta (*length*) e o número de divisões por aresta (*divisions*).

O algoritmo inicia calculando a dimensão de cada “quadrícula”, ao dividir o comprimento total da aresta pelo número de divisões. De seguida, através de dois ciclos aninhados, os eixos X e Z são percorridos para determinar as coordenadas dos vértices, mantendo a coordenada y sempre com o valor 0.

Para cada iteração, são gerados dois triângulos adjacentes que formam um quadrado (Figura 3). A ordem dos vértices segue o sentido anti-horário que é fundamental para a correta aplicação de *Backface Culling*. O resultado deste processo é a escrita sequencial das coordenadas no ficheiro `.3d` apresentado na Figura 3.

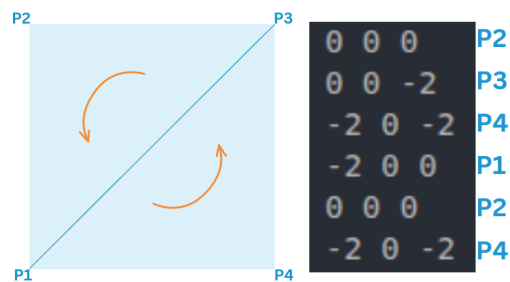


Figura 3 - Mecanismo de ordem dos vértices

1.2.2. *Box*

À semelhança do processo elaborado na primitiva anterior, a *Box* tem os mesmos parâmetros de entrada e segue quase a mesma lógica.

A implementação percorre as divisões solicitadas e, em cada iteração, calcula os vértices para as seis faces em simultâneo:

- Nas faces de frente e trás, a coordenada z é fixa enquanto x e y variam.
- Nas faces de cima e baixo, a coordenada y é fixa enquanto x e z variam;

- Nas faces laterais, a coordenada x é fixa enquanto y e z variam.

Tal como no plano, a ordem da inserção dos vértices é anti-horário para garantir que todas as faces apontem para o exterior do cubo.

1.2.3. Sphere

A primitiva *Sphere* recebe como parâmetros o raio da esfera e o número de *slices* e *stacks* pretendidas.

Para conseguirmos calcular as coordenadas dos pontos, recorreremos a um sistema de coordenadas esféricas, que, assumindo que o centro da esfera corresponde à origem, permite definir um ponto P da esfera por

$$P = (r, \theta, \varphi)$$

onde

- r representa o raio, isto é, a distância entre P e a origem,
- $\theta \in [0, 2\pi]$ é o ângulo azimutal, que representa o ângulo entre o eixo z positivo e a linha que une a origem com a projeção do ponto no plano xz ,
- $\varphi \in [0, \pi]$ é o ângulo polar, que representa o ângulo entre o eixo y e a linha que une o centro ao ponto.

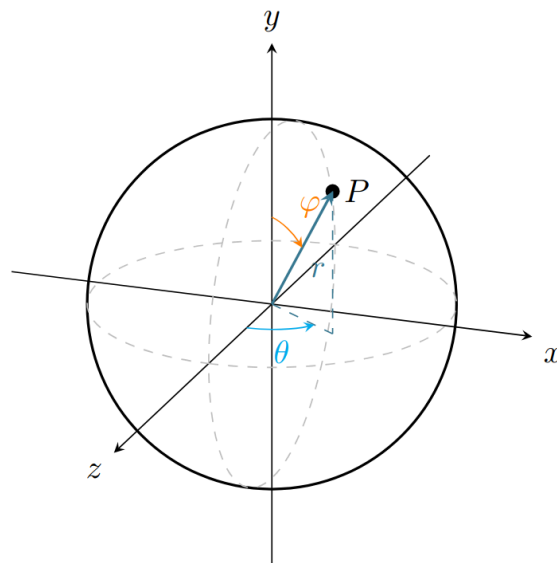


Figura 4 - Representação de coordenadas esféricas

A partir destes valores, é possível calcular as coordenadas cartesianas da seguinte forma:

$$\begin{aligned} x &= r \cos(\theta) \sin(\varphi) \\ y &= r \cos(\varphi) \\ z &= r \sin(\theta) \sin(\varphi) \end{aligned}$$

Para obter o valor de φ e θ para cada ponto, é necessário calcular os valores dos passos angulares, dividindo os intervalos do ângulo polar e do azimutal pelo número de *stacks* e *slices*, respetivamente.

$$\Delta\varphi = \frac{\pi}{n^{\circ}stacks}$$

$$\Delta\theta = \frac{2\pi}{n^{\circ}slices}$$

Assim, somando os valores incrementais obtém-se os possíveis valores de φ e θ :

$$\varphi_i = i \Delta\varphi, i \in [1, n^{\circ} stacks[$$

$$\theta_j = j \Delta\theta, j \in [1, n^{\circ} slices[$$

Por fim, para gerar os vértices, primeiro são definidos os pontos da esfera que intersectam o eixo y, comumente designados por polo norte e por polo sul, com as coordenadas $(0, r, 0)$ e $(0, -r, 0)$, respectivamente. De seguida, para cada φ_i são calculadas as coordenadas dos vértices para todos os valores de θ_j .

Com os vértices gerados, pode-se proceder à criação das faces. Para tal, para a stack superior, cada par de vértices adjacentes calculados para φ_1 formam um triângulo com o polo norte. O mesmo acontece para a última stack, onde os vértices $\varphi_{n^{\circ}stacks-1}$ são ligados ao polo sul.

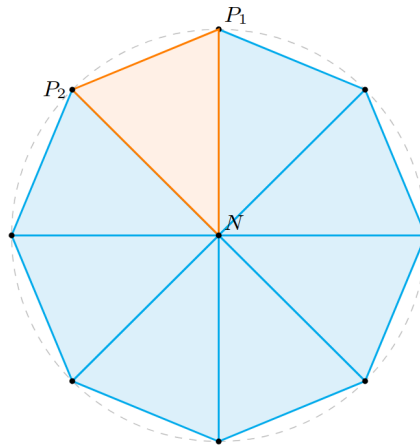


Figura 5 - Triângulos formados pelos vértices da primeira *stack* com o polo norte para 8 *slices*

Para as *stacks* intermédias, os triângulos constituem os quadriláteros formados pela interseção de uma *stack* e de uma *slice*. Considerando, para cada ponto gerado $P_{(i,j)}$, o ponto seguinte com o mesmo ângulo polar $P_{(i,j+1)}$, o ponto seguinte com o mesmo ângulo azimutal $P_{(i+1,j)}$ e o ponto com ambos os ângulos incrementados $P_{(i+1,j+1)}$, tem-se que os triângulos a formar as faces da esfera são dados por:

- $P_{(i,j)}, P_{(i+1,j+1)}, P_{(i+1,j)}$
- $P_{(i,j)}, P_{(i,j+1)}, P_{(i+1,j+1)}$

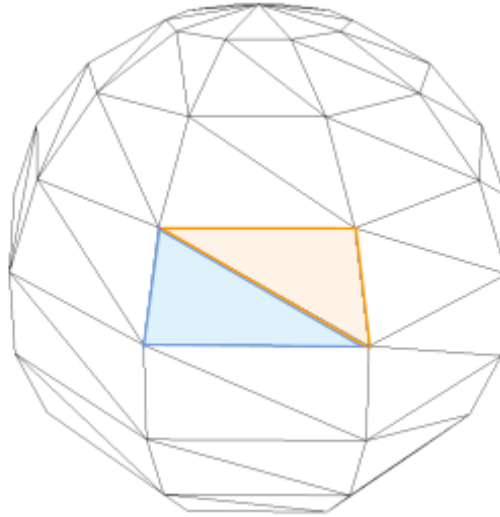


Figura 6 - Exemplo dos triângulos das stacks intermédias

1.2.4. Cone

A primitiva Cone recebe como parâmetros o raio da base (r), a altura do sólido (h) e o número de *slices* e *stacks* pretendidas.

Para a implementação do cone ser clara, é fundamental compreender o papel que as *slices* têm na representação do sólido. Como demonstrado na figura 2, as *slices* são divisões verticais no sólido. Assim, ao definir este número, define-se indiretamente, a figura geométrica da base da primitiva que se vai construir. Ou seja, quantos mais *slices* tiver, mais a base se aproxima de um círculo, como demonstrado na figura 7.

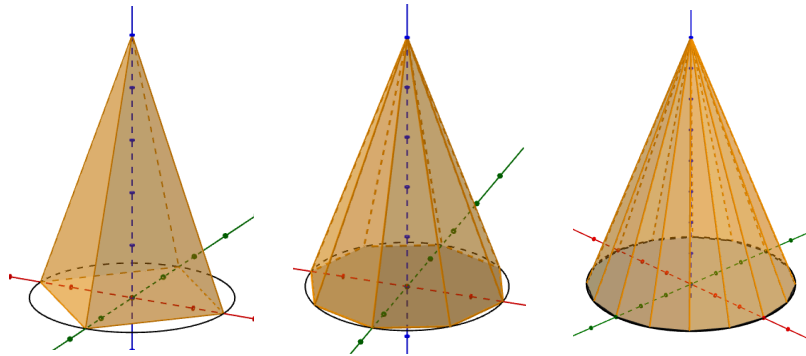


Figura 7.a - 4 *slices*

Figura 7.b - 8 *slices*

Figura 7.c - 16 *slices*

Figuras 7 - Cones com diferentes números de *slices*

Para calcular os pontos do sólido é necessário calcular, inicialmente, a altura de cada *stack* (h_s) e o ângulo de cada *slice* (α):

$$h_s = \frac{\text{altura do cone } (h)}{n^{\circ} \text{ stacks}}$$

$$\alpha = \frac{2\pi}{n^{\circ} \text{ slices}}$$

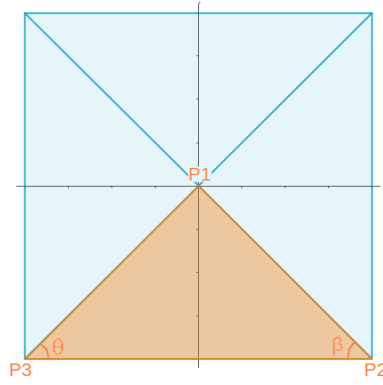


Figura 8 - Exemplo da base de um cone com 4 slices

A base está assente no plano xz , ou seja y é sempre 0. Para cada *slice* n , gera-se um triângulo definido por:

$$\begin{aligned} P1 &= (0, 0, 0) \\ P2 &= (r \sen \theta, 0, r \cos \theta) \\ P3 &= (r \sen \beta, 0, r \cos \beta) \end{aligned}$$

Sendo:

- $\beta = n * \alpha$
- $\theta = (n + 1) * \alpha$

As faces laterais são construídas percorrendo cada *stack* i e, dentro desta, cada *slice* j . O raio de cada nível é calculado linearmente:

$$\begin{aligned} r_{atual} \text{ (raio na altura mais baixa do stack)} &= r * \left(1 - \frac{i}{n^\circ \text{ de stacks}}\right) \\ r_{seguinte} \text{ (raio na altura mais alta do stack)} &= r * \left(1 - \frac{i+1}{n^\circ \text{ de stacks}}\right) \end{aligned}$$

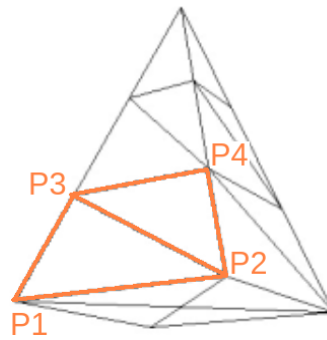


Figura 9 - Exemplo de uma secção numa face lateral

Cada secção lateral, à exceção da última, é dividida em dois: (P1, P2, P3) e (P2, P4, P3).

Para cada secção da face lateral, destacado na figura 9, calcula-se quatro pontos baseados nos ângulos β e θ :

$$\begin{aligned} P_1 &= (r_{atual} \sen \beta, h_1, r_{atual} * \cos \beta) \\ P_2 &= (r_{atual} \sen \theta, h_1, r_{atual} * \cos \theta) \\ P_3 &= (r_{seguinte} \sen \beta, h_2, r_{seguinte} * \cos \beta) \\ P_4 &= (r_{seguinte} \sen \theta, h_2, r_{seguinte} * \cos \theta) \end{aligned}$$

Sendo:

- h_1 : altura mais baixa da stack atual = $i * h_s$
- h_2 : altura mais alta da stack atual = $(i + 1) * h_s$
- $\beta = j * \alpha$
- $\theta = (j + 1) * \alpha$

Na stack mais alta ($i == n^\circ \text{ stacks} - 1$), é desenhado apenas um triângulo (P1, P2, V), em que o vértice do topo do cone é $V = (0, h, 0)$.

É importante mencionar que a ordem dos vértices no triângulo está definida de modo a que as normais apontem para o exterior do sólido, isto é, é fundamental que os vértices sejam guardados no sentido anti-horário.

1.2.5. Cylinder

A primitiva *Cylinder*, foi um extra implementado pelo grupo. Esta primitiva gera um cilindro com a base inferior assente no eixo XZ e estende-se verticalmente até a coordenada correspondente à sua altura. Esta primitiva recebe como parâmetros o raio da base (r), a altura do cilindro (h) e número de slices (s) em que a superfície curva será dividida (Figura 10).

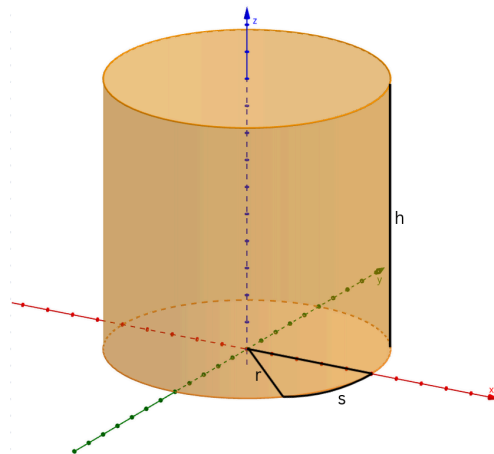


Figura 10 - Parâmetros de entrada do Cilindro

Consideremos o ângulo α como sendo a rotação em torno do eixo vertical. As coordenadas cartesianas (X e Z) dos vértices ao longo da superfície circular do cilindro são calculadas com recurso a funções geométricas:

$$\begin{aligned} x &= r * \sin(\alpha) \\ z &= r * \cos(\alpha) \end{aligned}$$

A coordenada Y, por sua vez, assume apenas dois valores fixos: 0 para definir os pontos da base inferior e h para os pontos da base superior.

Para garantir divisão uniforme da superfície da curva, o incremento angular a cada iteração ($\Delta\alpha$) é obtido dividindo a circunferência pelo número de slices:

$$\Delta\alpha = \frac{2\pi}{n^\circ \text{slices}}$$

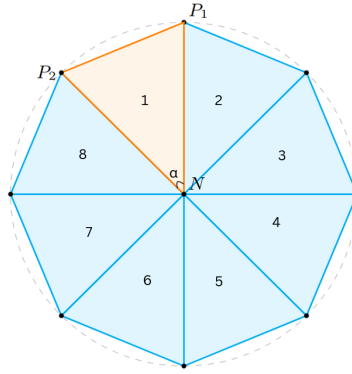


Figura 11 - Bases o cilindro

Assim, a construção do cilindro é feita fatia a fatia, onde em cada iteração do ciclo calculam-se os pontos correspondentes ao ângulo atual (α) e ao ângulo seguinte ($\alpha + \Delta\alpha$). Como podemos ver na Figura 12, são gerados 4 triângulos que compõem uma fatia completa da primitiva:

- **Base inferior:** triângulo que une a origem da base $(0,0,0)$ aos dois pontos correspondentes no limite do círculo inferior.
- **Lateral:** 2 triângulos adjacentes que formam um retângulo curvo unindo dois pontos da base inferior aos dois pontos correspondentes na base superior.
- **Base superior:** triângulo que liga o centro do topo do cilindro $(0,h,0)$ aos dois pontos no limite do círculo superior.

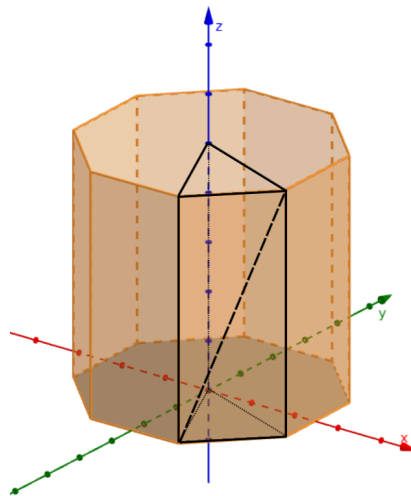


Figura 12 - Divisão de uma slice em triângulos

1.2.6. Torus

Assim como o cilindro, a primitiva *Torus* foi um extra implementado pelo grupo. Esta primitiva recebe como parâmetros a distância do centro do tubo ao centro do torus - o raio maior (R) -, o raio do tubo ou raio menor (r), o número de *slices* e *stacks* pretendidas.

Consideremos o ângulo $\alpha \in [0, 2\pi]$ como sendo a rotação de um ponto em torno do eixo central do torus e o ângulo $\beta \in [0, 2\pi]$ como a rotação em torno da circunferência de raio R que corresponde ao centro do tubo. Então, temos que as coordenadas cartesianas de um ponto no *torus* são dadas por:

$$x = (-R + r \cos(\beta)) \cos(\alpha)$$

$$y = r \cos(\beta)$$

$$z = (-R + r \cos(\beta)) \sin(\alpha)$$

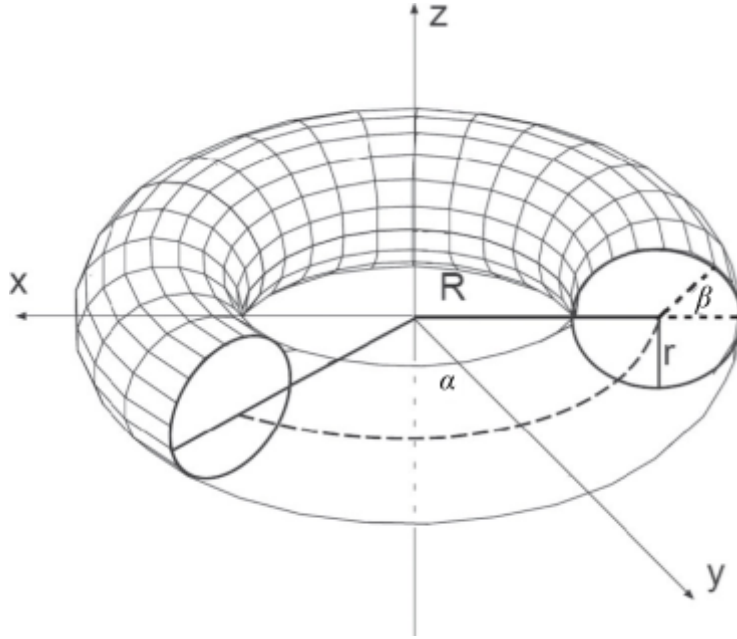


Figura 13 - Representação de um torus ¹

De forma semelhante à esfera, os valores incrementais necessários para obter o valor dos ângulos α e β para cada ponto são obtidos dividindo os intervalos a que pertencem pelo número de *slices* e *stacks*, respetivamente:

$$\Delta\alpha = \frac{2\pi}{n^{\circ}slices}$$

$$\Delta\beta = \frac{2\pi}{n^{\circ}stacks}$$

Logo, os valores possíveis de φ e θ para os pontos gerados serão dados por:

$$\alpha_i = i \Delta\alpha, i \in [0, n^{\circ}slices[$$

$$\beta_j = j \Delta\theta, j \in [0, n^{\circ}stacks[$$

Tendo o conjunto de todos os vértices da primitiva gerados, será necessário agrupá-los e ordená-los devidamente para a formação dos triângulos que vão constituir as faces do *torus*. Semelhantemente à esfera, se considerarmos para cada ponto gerado $P_{(i,j)}$ o ponto seguinte com o mesmo valor de β , $P_{(i,j+1)}$, o ponto seguinte com o mesmo ângulo α , $P_{(i+1,j)}$ e o ponto com ambos os ângulos incrementados, $P_{(i+1,j+1)}$, temos que os triângulos que formam as faces do torus são dados por:

- $P_{(i,j)}, P_{(i+1,j+1)}, P_{(i+1,j)}$
- $P_{(i,j)}, P_{(i,j+1)}, P_{(i+1,j+1)}$

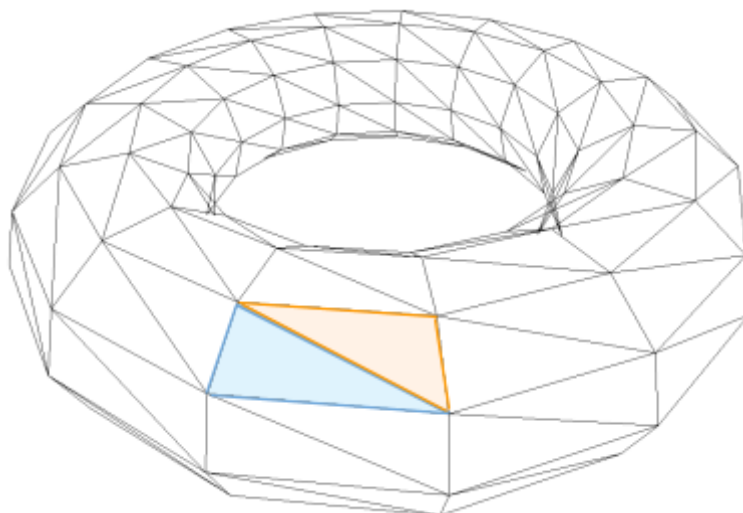


Figura 14 - Exemplo dos triângulos formados

2. Engine

O engine foi concebido para permitir a visualização das primitivas geradas anteriormente, utilizando a *API* gráfica *OpenGL*. A aplicação atua como um interpretador de cenas, recebendo como argumento o caminho de configuração com extensão *.xml*.

Para iniciar a aplicação utiliza-se o seguinte comando:

```
./engine <caminho_ficheiro_xml>
```

2.1. Parsing e Estrutura de Dados

O sistema de carregamento de dados divide-se em duas etapas distintas: a leitura e processamento da estrutura da cena (*.xml*) e a leitura dos dados geométricos (*.3d*). O ficheiro de configuração tem uma estrutura bem definida, contendo, nesta primeira parte:

- *Window*: Tamanho da janela a ser criada;
- *Camera*: Configuração da câmara;
- *Models*: Caminho dos ficheiros *.3d* gerados pelo *generator*.

2.1.1. Point

A estrutura *Point* é a unidade fundamental da representação geométrica, armazenando as coordenadas cartesianas (*x*, *y*, *z*) de um vértice.

2.1.2. Parsing do Ficheiro XML

Para otimizar o processo de leitura do ficheiro XML, que contém informações necessárias à renderização da cena, recorreu-se à biblioteca externa *RapidXML*. Os dados extraídos do ficheiro de configuração são guardados numa estrutura de dados centralizada, a classe *Scene*.

2.1.2.1. Scene

A classe *Scene* encapsula toda a descrição do ambiente virtual, organizando as informações em estruturas de dados dedicadas:

- **Window**: objeto que armazena a largura e altura da janela;
- **Camera**: objeto que armazena a posição, orientação e os parâmetros de projeção (*FOV*, *near* e *far planes*);
- **Models**: um vetor de vetores de objetos do tipo *Point*. Para cada modelo encontrado, é lido o ficheiro *.3d* na segunda parte do *parsing*.

2.1.3. Parsing do Ficheiro *.3d*

Uma vez obtido o caminho do ficheiro *.3d*, é executada a sua leitura para carregar os modelos para a memória.

O processo é feito lendo um linha por vez, criando-se um objeto *Point* por vértice e armazenando-o num vetor. No final da leitura, este vetor (que representa o modelo completo) é inserido no vetor de modelos da classe *Scene*.

2.2. Renderização da Cena

A renderização é a fase onde o motor percorre as estruturas de dados preenchidas durante o parsing dos ficheiros e utiliza a API gráfica *OpenGL* para desenhar os modelos no ecrã.

2.2.1. Configuração de Visualização

Para garantir os formatos visuais corretos, o motor define o estado do contexto gráfico antes de proceder ao desenho de cada frame. Para utilizar a técnica do *Back-Face Culling*, são utilizadas as funções `glEnable(GL_CULL_FACE)` e `glCullFace(GL_BACK)` para instruir a placa gráfica a ignorar as faces traseiras do sólido.

2.2.2. Desenho Final

A renderização final da cena segue uma sequência lógica que transforma os dados brutos lidos dos ficheiros numa representação visual.

1. **Preparação da cena**: O processo inicia-se com o parsing do ficheiro XML e o carregamento de todos os ficheiros *.3d* associados. Toda esta informação é armazenada num objeto *Scene*.
2. **Configuração da câmara**: Antes de desenhar qualquer geometria, a câmara é posicionada utilizando os parâmetros de posição, foco e orientação guardados.
3. **Referencial Cartesiano**: São desenhados, primeiro, os eixos centrados na origem (0, 0, 0). Estes eixos são coloridos para facilitar a distinção: o eixo X é representado a vermelho, o Y a verde e o Z a azul.
4. **Desenho de modelos**: Por fim, a lista de modelos carregados é percorrida. Cada modelo é processado sequencialmente, onde o sistema interpreta os conjuntos de vértices para desenhar triângulo a triângulo.

Conclusão

Com a implementação destas aplicações, foi possível cumprir os objetivos da primeira fase do projeto. O *Generator* é capaz de criar primitivas geométricas complexas, enquanto o *Engine* interpreta estas configurações e executa-as num ambiente 3D.

Estas estruturas servem de base sólida para as fases posteriores do projeto.

Referências e Bibliografia

¹Kneer, F., Schöll, E., & Dahlem, M. A. (2014). Nucleation of reaction-diffusion waves on curved surfaces. *New Journal of Physics*, 16(5), 053010.

<https://doi.org/10.1088/1367-2630/16/5/053010>

Cooper, G. R. J. (2002). Dynamics on the complex sphere and torus. *Computers & Graphics*, 26(1), 151–154. [https://doi.org/10.1016/S0097-8493\(01\)00163-7](https://doi.org/10.1016/S0097-8493(01)00163-7)